

Monte Carlo Prediction: First-Visit vs Every-Visit

A Visual Guide with Worked Examples

CSCE 775 – Reinforcement Learning

I. INTRODUCTION

In reinforcement learning, one of the most fundamental questions is: “How good is it to be in a particular state?” We answer this by estimating the **value function** $V(s)$, which tells us the total reward we can expect to collect starting from state s if we follow a particular policy (i.e., a particular strategy for choosing actions). A high value means the state is a good place to be; a low or negative value means things are likely to go poorly from there.

A. Two Approaches: Dynamic Programming vs Monte Carlo

There are two major families of methods for estimating $V(s)$.

Dynamic Programming (DP) uses the Bellman equation to compute $V(s)$ by looking at all the states we *could* transition to, weighted by their probabilities. This requires a **model** of the environment, meaning we need to know two things exactly. First, the *transition probabilities* $p(s' | s, a)$, which answer the question “if I’m in state s and take action a , what is the probability I end up in state s' ?” Second, the *reward function* $r(s, a, s')$, which tells us the reward for each transition. If we have these, DP can compute the value function precisely without ever actually interacting with the environment.

The problem is that in many real-world situations, we *do not* have this model. Consider a robot navigating a warehouse, or an agent playing a video game. We might not know the exact probabilities of what happens next. We only know what *actually happened* when we tried things.

Monte Carlo (MC) prediction takes a completely different approach. Instead of computing values from a model, MC methods learn directly from **experience**. We let the agent interact with the environment, collect complete episodes (sequences of states, actions, and rewards from start to finish), and then estimate $V(s)$ by averaging the actual rewards we observed. This is what it means to be **model-free**: we do not need to know $p(s' | s, a)$ or $r(s, a, s')$ at all. We just need to be able to generate episodes by running the policy.

When to Use Which?

Use Dynamic Programming when you have a complete, accurate model of the environment (transition probabilities and rewards). DP is exact and efficient for small, well-defined problems like simple gridworlds where all the rules are known.

Use Monte Carlo methods when you do not have a model, or the model is too complex to use. MC only requires the ability to generate sample episodes. This makes MC applicable to situations like games, robotics, or any setting where you can simulate or interact with the environment but cannot write down all the transition probabilities.

B. The Core Idea

The value of a state under a policy π is defined as the *expected return* starting from that state.

$$V^\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s] \quad (1)$$

Let us unpack each piece of notation. $V^\pi(s)$ is the value of state s when following policy π . The symbol $\mathbb{E}_\pi[\cdot]$ means “the expected value (average) if we follow policy π .” The term G_t is the **return** at time step t , which is the total discounted reward from time t onward (we will compute this explicitly in Section IV). The expression $S_t = s$ means “given that we are in state s at time t .” Putting it together, Equation (1) says: the value of state s is the average total reward we would get, starting from s and following our policy.

MC prediction approximates this expected value by collecting actual returns G_t from many episodes and computing their *sample mean* (just a plain average). The two most common variants, **first-visit MC** and **every-visit MC**, differ in how they handle states that appear more than once in a single episode. The rest of this guide walks through a concrete example to make this difference crystal clear.

II. THE GRIDWORLD

Consider the following 3×3 gridworld with states S_1 through S_9 . The agent can move in the four cardinal directions. State S_9 is the **terminal state** (goal). Stepping into the goal yields a reward of $R = +10$, while every other step yields $R = -1$. The discount factor is $\gamma = 0.9$.

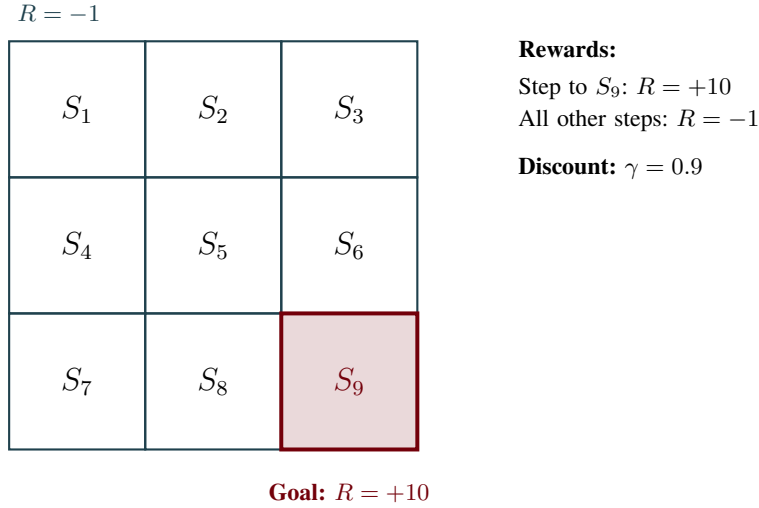


Fig. 1. The 3×3 gridworld. S_9 (garnet) is the terminal goal state.

III. A SAMPLE EPISODE

Suppose the agent follows its policy and generates the following episode. Notice that the agent revisits states S_1 and S_2 , which is exactly the scenario where first-visit and every-visit MC differ.

Episode Trajectory

$S_1 \xrightarrow{-1} S_2 \xrightarrow{-1} S_5 \xrightarrow{-1} S_4 \xrightarrow{-1} S_1 \xrightarrow{-1} S_2 \xrightarrow{-1} S_3 \xrightarrow{-1} S_6 \xrightarrow{+10} S_9$

States shown in **rose** are *revisits*, i.e., the agent has already visited them earlier in the episode.

The following diagram traces this path on the gridworld. Each arrow is numbered by its time step, and revisited transitions are drawn in **rose**.

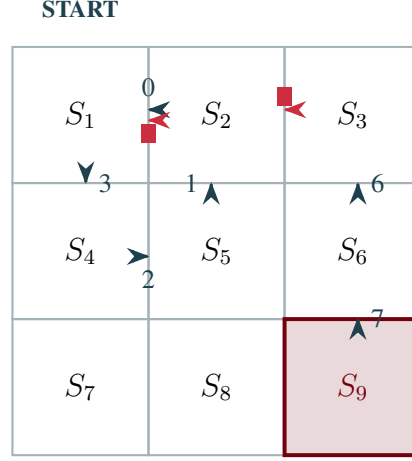


Fig. 2. Episode path on the gridworld. Black arrows show first visits; **rose arrows** show revisits of S_1 and S_2 .

IV. COMPUTING RETURNS

The return G_t at each time step is computed *backward* from the terminal state using the recursive definition.

$$G_t = R_{t+1} + \gamma G_{t+1} \quad (2)$$

Starting from the final transition at $t = 7$ and working backward gives the following.

Backward Return Computation

$G_7 = R_8 = +10$	($S_6 \rightarrow S_9$: terminal reward)
$G_6 = R_7 + \gamma G_7 = -1 + 0.9(10) = 8.0$	(at S_3)
$G_5 = R_6 + \gamma G_6 = -1 + 0.9(8.0) = 6.2$	(at S_2 , 2nd visit)
$G_4 = R_5 + \gamma G_5 = -1 + 0.9(6.2) = 4.58$	(at S_1 , 2nd visit)
$G_3 = R_4 + \gamma G_4 = -1 + 0.9(4.58) = 3.122$	(at S_4)
$G_2 = R_3 + \gamma G_3 = -1 + 0.9(3.122) = 1.810$	(at S_5)
$G_1 = R_2 + \gamma G_2 = -1 + 0.9(1.810) = 0.629$	(at S_2 , 1st visit)
$G_0 = R_1 + \gamma G_1 = -1 + 0.9(0.629) = -0.434$	(at S_1 , 1st visit)

The summary of all returns is given in Table I.

TABLE I
RETURNS G_t FOR EACH TIME STEP IN THE EPISODE.

Time t	State	Return G_t	Visit #
0	S_1	-0.434	1st
1	S_2	0.629	1st
2	S_5	1.810	1st
3	S_4	3.122	1st
4	S_1	4.58	2nd
5	S_2	6.2	2nd
6	S_3	8.0	1st
7	S_6	10.0	1st

V. FIRST-VISIT MONTE CARLO PREDICTION

Definition: First-Visit MC

For each state s appearing in an episode, record the return G_t only for the **first time** s is visited in that episode. The value estimate is the average of these first-visit returns across all episodes.

$$V(s) \approx \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_t^{(i)} \quad \text{where each } G_t^{(i)} \text{ is from the first visit to } s \text{ in episode } i \quad (3)$$

In our episode, first-visit MC uses only the returns from $t = 0$ and $t = 1$ for S_1 and S_2 respectively, *ignoring* the second visits at $t = 4$ and $t = 5$.

First-Visit MC Pseudocode

$V(s) \leftarrow 0$, Returns(s) $\leftarrow$ empty list, for all s

Generate episode: $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T$

$G \leftarrow 0$

$t = T - 1, T - 2, \dots, 0$:

$G \leftarrow R_{t+1} + \gamma G$

S_t not in $\{S_0, S_1, \dots, S_{t-1}\}$:

Append G to Returns(S_t)

$V(S_t) \leftarrow \text{average}(\text{Returns}(S_t))$

$\triangleleft$ first visit only

The following grid shows the first-visit value estimates after processing this single episode. Second-visit returns are crossed out to emphasize they are discarded.

S_1 -0.434 4.58	S_2 0.629 6.2	S_3 8.0
S_4 3.122	S_5 1.810	S_6 10.0
S_7 —	S_8 —	S_9 Terminal

Fig. 3. First-visit MC estimates. ~~Struck values~~ are second-visit returns that are discarded.

VI. EVERY-VISIT MONTE CARLO PREDICTION

Definition: Every-Visit MC

For each state s appearing in an episode, record the return G_t for **every time** s is visited. The value estimate is the average of *all* observed returns for s across all episodes.

$$V(s) \approx \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_t^{(i)} \quad \text{where each } G_t^{(i)} \text{ is from any visit to } s \quad (4)$$

Every-visit MC uses *both* returns for S_1 and S_2 .

For S_1 :

$$V(S_1) = \frac{G_0 + G_4}{2} = \frac{-0.434 + 4.58}{2} = 2.073 \quad (5)$$

For S_2 :

$$V(S_2) = \frac{G_1 + G_5}{2} = \frac{0.629 + 6.2}{2} = 3.414 \quad (6)$$

S_1 $\frac{-0.434+4.58}{2}$ 2.073	S_2 $\frac{0.629+6.2}{2}$ 3.414	S_3 8.0
S_4 3.122	S_5 1.810	S_6 10.0
S_7 —	S_8 —	S_9 Terminal

Fig. 4. Every-visit MC estimates. Both returns are averaged for S_1 and S_2 .

VII. SIDE-BY-SIDE COMPARISON

The two grids below place the first-visit and every-visit estimates next to each other. The cells where the estimates differ (S_1 and S_2) are highlighted in **rose**.

First-Visit MC			Every-Visit MC		
S_1 −0.434	S_2 0.629	S_3 8.0	S_1 2.073	S_2 3.414	S_3 8.0
S_4 3.122	S_5 1.810	S_6 10.0	S_4 3.122	S_5 1.810	S_6 10.0
S_7	S_8	S_9	S_7	S_8	S_9

Fig. 5. Side-by-side comparison. Cells highlighted in **rose** differ between the two methods.

TABLE II
NUMERICAL COMPARISON OF FIRST-VISIT AND EVERY-VISIT MC ESTIMATES.

State	First-Visit $V(s)$	Every-Visit $V(s)$	Difference
S_1	−0.434	2.073	+2.507
S_2	0.629	3.414	+2.785
S_3	8.0	8.0	0
S_4	3.122	3.122	0
S_5	1.810	1.810	0
S_6	10.0	10.0	0

The difference is substantial. First-visit MC gives $V(S_1) = -0.434$ while every-visit MC gives $V(S_1) = 2.073$. This is because the second visit to S_1 (at $t = 4$) occurs closer to the goal and thus has a much higher return ($G_4 = 4.58$). Every-visit MC includes this optimistic return in its average, pulling the estimate upward.

VIII. KEY TAKEAWAYS

Bias and Variance Properties**First-Visit MC:**

- Each return G_t is an *unbiased* estimate of $V^\pi(s)$ because the first visit to s is a fresh sample from the stationary distribution of episodes starting from s .
- Returns across different episodes are *independent*, which simplifies convergence analysis.
- Converges to $V^\pi(s)$ by the strong law of large numbers as the number of episodes $\rightarrow \infty$.

Every-Visit MC:

- Technically *biased* for finite samples because returns from multiple visits within the same episode are correlated (later visits depend on the path that led to the revisit).
- However, the bias is *asymptotically zero* and every-visit MC is *consistent*, converging to $V^\pi(s)$ as episodes $\rightarrow \infty$.
- Generally has *lower variance* than first-visit MC because it uses more data points per episode.
- Often preferred in practice due to simpler implementation (no need to track which states have been visited).

When Does It Matter?

The difference between first-visit and every-visit MC is most pronounced when:

- Episodes are long and contain many state revisits.
- Only a small number of episodes are available.
- The returns from different visits to the same state vary significantly (as in our example, where $G_0 = -0.434$ and $G_4 = 4.58$ for S_1).

As the number of episodes grows, both methods converge to the same true value function $V^\pi(s)$ and the practical difference vanishes.